

“A Watermark for Large Language Models” by John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein addresses the growing challenge of distinguishing human-written text from AI-generated content as large language models become more widely used. The paper proposes a watermarking method that embeds detectable signals into generated text by subtly biasing token selection during the generation process.

Chosen Result

We aim to reproduce three figures from the paper. Figures 3a and 3b show how z-score varies with γ and δ , where δ is watermark strength, γ is the fraction of tokens in the green list, and z-score measures how many standard deviations the observed green-token frequency is above expectation. Figure 2 (left) shows the relationship between perplexity and z-score for varying δ , where lower perplexity indicates better text quality. We selected these figures to analyze the trade-off between watermark strength, detectability, and generation quality.

Methodology

Our re-implementation approach follows the following steps:

1. Data cleaning
We extract the raw text from allenai/c4 then parse each text into their respective array of token_ids. We then only keep text whose token_ids size are within 250 to 400.
2. Green list generation
To create the green list, we use the previous_token_id, gamma, and the vocabulary size. First, the previous_token_id is used to generate a deterministic seed, which ensures the same previous token always produces the same random permutation of the vocabulary. Then, we select the first vocab_size * gamma token IDs from that permutation as the green list.
3. Watermark Generation
To watermark a response, we first pass the current sequence into the model to obtain the logits over the entire vocabulary for the next token prediction. Using the last generated token, we construct a green list. The logits corresponding to tokens in the green list are increased by a bias value (delta), making those tokens more likely to be sampled. After applying softmax, the next token is sampled from the new distribution. This process is repeated for each newly generated token throughout generation.
4. Watermark Detection
To detect a watermark, we reconstruct the green list for each token using the previous token and count how many generated tokens fall inside the green list. A statistical z-score test is then applied to measure whether the number of green tokens is significantly higher than random chance. A z-score of 4 means the result is 4 standard deviations above the expected mean, and in this paper, a z-score of 4 or greater is classified as AI-generated text. (See figure 4 for the z-score formula)

Evaluation Metrics

Our evaluation metrics are the z-score and perplexity. The z-score is computed per sequence by counting how many tokens fall into the reconstructed green list and normalizing by sequence length, where the expected mean corresponds to the number of green tokens under random

sampling given the green list proportion γ . A z-score of 4 means the result is 4 standard deviations above this expected mean, and in this paper a z-score of 4 or greater is classified as AI-generated text (see Figure 4 for the z-score formula). Perplexity is computed using a reference OPT language model as the exponentiated cross-entropy loss over the generated tokens conditioned on the prompt, where lower values indicate more natural and higher-quality text. We include perplexity because watermarking can potentially degrade generation quality.

Results & Analysis

Our re-implementation matches the paper’s main trends in Figures 3a, 3b, and Figure 2 (left). Increasing δ raises the z-score, improving watermark detectability, but also increases perplexity, indicating reduced text quality. Decreasing γ generally increases z-score while also increasing perplexity, since a smaller green list strengthens the statistical signal but constrains generation. Our main quantitative discrepancy is higher perplexity compared to the paper, which we attribute to using the smaller OPT-350M model instead of OPT-6.7B, as smaller models naturally produce lower-quality outputs.

In the context of the paper’s contribution, our results support the main claim that watermarking can embed detectable signals in LLM outputs without model access. More broadly, they reinforce the trade-off between detectability and text quality, and show that performance is sensitive to model size and decoding parameters.

Reflections

Roadblocks Encountered:

One major challenge we faced was batching text inputs with variable token lengths. Since each text sample produced a different number of token IDs after tokenization, we needed a way to efficiently batch them together. To solve this, we padded every input sequence to a fixed maximum length of 400 tokens and used an attention mask to indicate which indices contained valid tokens versus padding.

Another significant challenge was debugging results that differed greatly from those reported in the paper. After extensive debugging, we discovered that our watermark generation loop continued generating repeated `<eos>` tokens until the loop termination condition was met. Initially, we assumed generation would stop immediately after the first `<eos>` token appeared. As a result, these repeated `<eos>` tokens were mistakenly included in our statistical analysis, leading to incorrect experimental results.

Extensions:

For our extension, we tested to see if our watermark still works after translating a watermarked response to German then back to English. To our surprise it survived, most interestingly it remained detectable starting at $\delta = 2.0$ (Figure B2) which is the value the paper chose for its parameter, despite not having done this experiment. Additionally we tested what would happen when parameters were varied (Figures B1-4) to determine if the paper had the optimal selection for delta, gamma, and threshold, which we discovered it did.

References

- Original Paper: <https://arxiv.org/pdf/2301.10226>
- Models used: 'facebook/opt-350m'
- Dataset used: 'allenai/c4'
- Translation models used: 'Helsinki-NLP/opus-mt-en-de', 'Helsinki-NLP/opus-mt-de-en'
- Frameworks used: PyTorch, Hugging Face Transformers, Hugging Face Datasets, NumPy, Pandas, Matplotlib

Paper Figures

Figure 3a and 3b

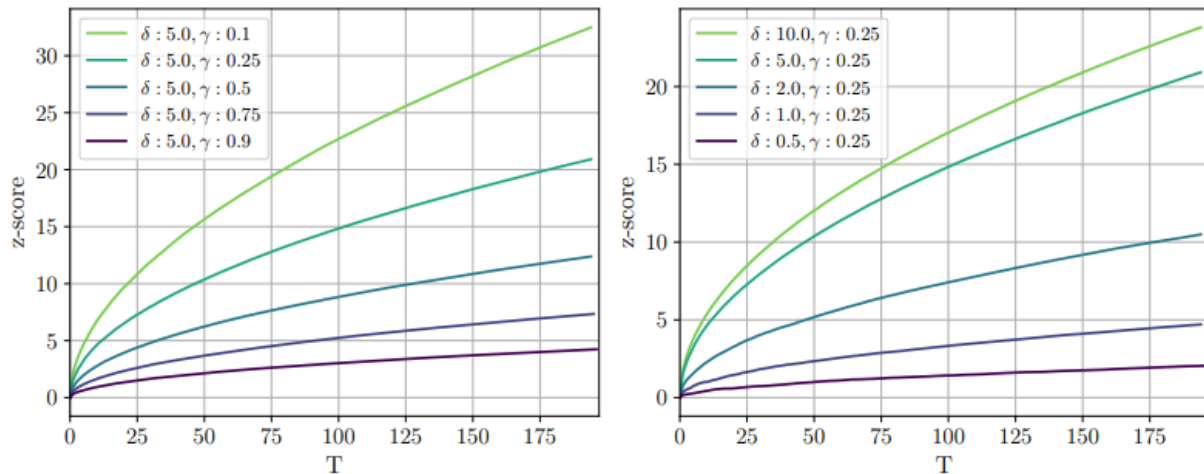


Figure 2 (left)

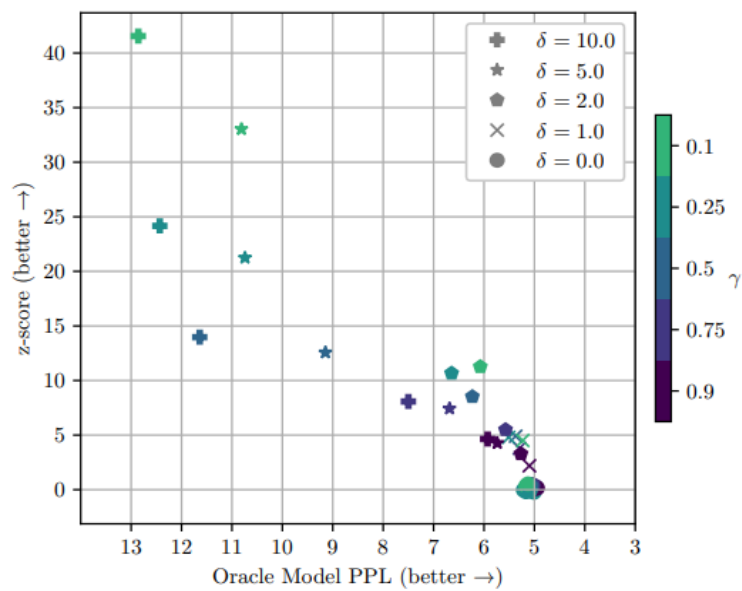


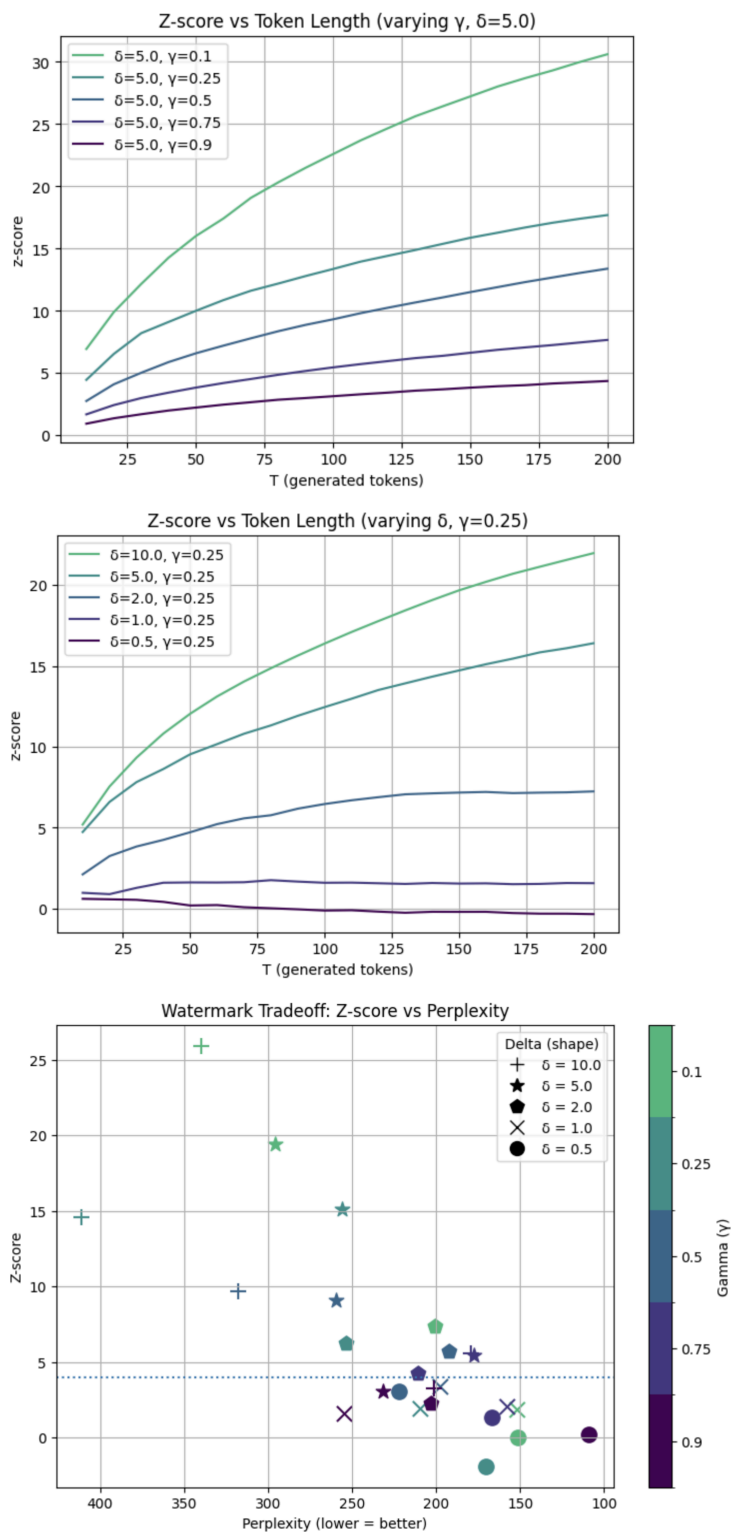
Figure 4

$$z = (|s|_G - \gamma T) / \sqrt{T\gamma(1 - \gamma)}.$$

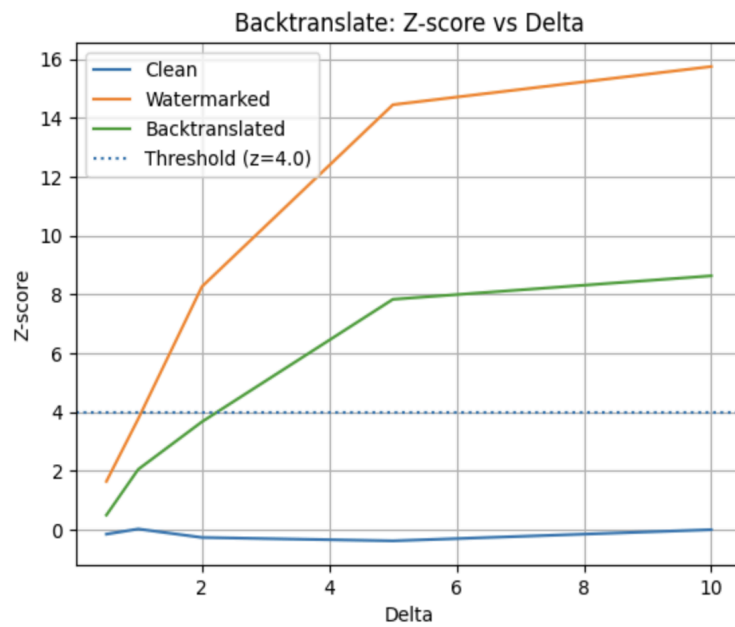
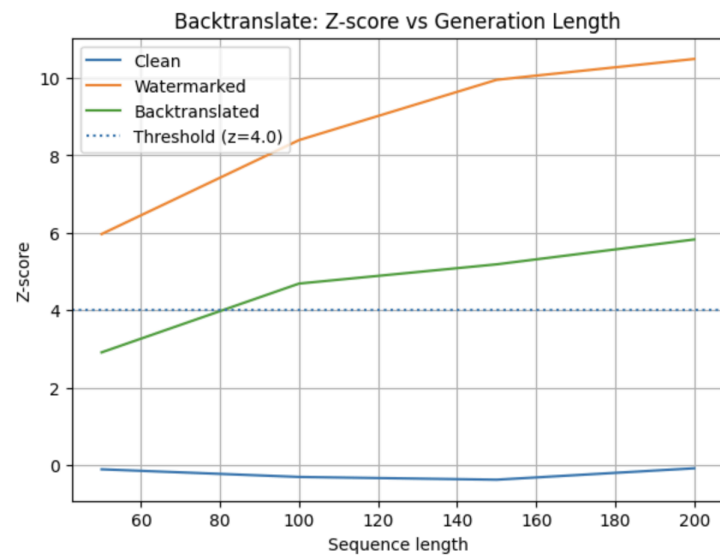
Where:

- $|s|_G$: size of green list
- T : number of tokens in response
- γ : fraction of the vocabulary assigned to the green list

Reimplementation Figures (3a, 3b, 2 (left))



Extension Figures (B1, B2)



Extension Figures Cont. (E1, E2, E3, E4)

